CareerPilot - Stage 1 Design Report

EECS3311 Software Design | Muhammad Abbas Kermali | October 9, 2026

1. Project overview and architecture

CareerPilot helps students and early-career applicants prepare job applications. It compares their experience with a posting, helps draft documents, provides interview practice and tracks applications. The aim is to reduce repeated work while keeping the applicant in control of factual claims and saved documents.

The application will use Java, a Swing GUI and a CLI. Both interfaces call CareerAssistantFacade.execute(), which routes requests to services for profiles, jobs, analysis, documents, interviews, applications, plans and export. Services use injected LLMClient, WorkspaceRepository and DocumentParser interfaces. Local JSON files store records with IDs, revisions and source links. GUI model calls run outside the UI event thread.

The planned model is GPT-4.1 mini, accessed through OpenAILLMAdapter [R10]. It extracts information, compares requirements and drafts text. Java handles validation, state, dates, storage and export. Supported inputs are text-based PDF/TXT resumes and pasted/TXT job postings. Export produces Markdown or plain text. OCR, scraping, email sending and automatic application submission are outside the scope.

Agent behaviour

For application preparation, CareerAgent.run() chooses tools based on the goal, saved work and observations from earlier calls. It can reuse a current resume, draft a missing letter, refresh stale analysis, ask about missing evidence or revise an unrealistic plan. This makes it a multi-step agent rather than a single drafting prompt.

Tool	Purpose / receiving class
getProfile / getJob	GetProfileTool / GetJobTool retrieve reviewed facts and requirements.
analyzeFit	AnalyzeFitTool calls AnalysisService to compare requirements and evidence.
inspectDocuments	InspectDocumentsTool checks missing, current or stale documents.
draftApplication	DraftApplicationTool creates an unsaved resume or letter preview.
validatePlan	ValidatePlanTool checks references, task dependencies and available time.

ToolRegistry checks the tool name, arguments and workspace scope before binding an InvokeToolCommand. Its result becomes context for the next model decision. Runs stop after 8 tool calls or 60 seconds; malformed output gets at most one repair attempt. PlanService checks the final plan again. The user separately approves plans and document drafts.

Saved records provide memory. Changes to source revisions make earlier analyses stale. Uploaded text is treated as data, not instructions. API credentials stay outside source code, and logs contain timing/outcomes rather than personal text. Evidence links help review but do not guarantee that a generated claim is accurate.

2. Features and use cases: F01-F04

Actor: Applicant; AI provider supports hybrid features. Trigger: the GUI action or CLI command shown. Fxx maps to UCxx. The numbered workflow is the main success scenario and explains class collaboration.

F01 / UC01 - Import resume and extract profile

Goal: Extract a reviewable profile from a resume. **Type:** Hybrid.

Input / preconditions: Readable PDF/TXT resume, up to 5 MB; application running.

GUI: Profile > Import Resume, review, Save. **CLI:** career resume import --file resume.pdf

Workflow: 1. ProfileService.importResume() selects a ResumeImporter. importText() uses createParser() and extractText(). 2. LLMClient.generate() extracts fields with source evidence. 3. The user reviews the preview; updateProfile() saves the profile and resume atomically.

Alternatives/errors: Reject unsupported, encrypted, scanned-only or empty files. Failed extraction or cancellation leaves records unchanged.

Output / postconditions: Confirmed CandidateProfile and source Resume are saved. **Sequence:** SD01, SD08.

F02 / UC02 - Review and correct candidate profile

Goal: Correct skills, experience and education. **Type:** Deterministic.

Input / preconditions: Saved profile ID and edited fields; a profile must exist.

GUI: Profile > Edit, then Save. **CLI:** career profile edit

Workflow: 1. ProfileService.getProfile() loads the record. 2. updateProfile() validates edited fields and the expected revision. 3. WorkspaceRepository.save() stores the update; earlier analyses retain source revisions and become stale.

Alternatives/errors: Reject invalid dates/required fields. A revision conflict preserves edits and requests reload.

Output / postconditions: Updated CandidateProfile is saved with a new revision. **Sequence:** SD01.

F03 / UC03 - Import and analyze a job description

Goal: Extract job requirements with source quotes. **Type:** Hybrid.

Input / preconditions: Posting text, optional company/URL/deadline; non-empty input and AI access.

GUI: Jobs > Add Job, review, Save. **CLI:** career job import --file posting.txt

Workflow: 1. JobService.analyzeJob() calls LLMClient.generate(). 2. The service checks the schema and source quotes. 3. The user reviews requirements and details; saveJob() persists the reviewed posting.

Alternatives/errors: Reject empty input; keep unknown deadlines blank. AI failure preserves entered text for retry.

Output / postconditions: Reviewed JobPosting and requirements are saved. **Sequence:** SD02, SD08.

F04 / UC04 - Analyze job fit with supporting evidence

Goal: Compare each requirement with profile evidence. **Type:** Hybrid.

Input / preconditions: Confirmed profile and reviewed job IDs; both records must exist.

GUI: Select job > Analyze Fit. **CLI:** career job fit --job J1

Workflow: 1. AnalysisService.analyzeFit() loads the profile and job. 2. LLMClient.generate() compares requirements and evidence. 3. The service validates references and saves supported, partial or not-evidenced findings with source revisions.

Alternatives/errors: Reject unknown IDs; repair invalid evidence once. Missing evidence does not prove a missing skill.

Output / postconditions: FitReport is saved and displayed; it does not predict hiring probability. **Sequence:** SD02, SD08.

2. Features and use cases: F05-F08

Actor: Applicant; AI provider supports hybrid features. Trigger: the GUI action or CLI command shown. Fxx maps to UCxx. The numbered workflow is the main success scenario and explains class collaboration.

F05 / UC05 - Identify gaps and suggest next steps

Goal: Turn gaps into suggested next steps. **Type:** Hybrid.

Input / preconditions: Existing profile/job and current fit report, or enough data to generate one.

GUI: Select job > Skill Gaps. **CLI:** career job gaps --job J1

Workflow: 1. AnalysisService.identifyGaps() loads a current FitReport or calls analyzeFit(). 2. LLMClient.generate() suggests actions for partial or not-evidenced requirements. 3. The service validates requirement links and saves the report.

Alternatives/errors: Recompute stale analysis. No gaps produces an empty report; AI failure preserves the dated previous report.

Output / postconditions: GapReport separates missing evidence from learning needs. **Sequence:** SD02, SD08.

F06 / UC06 - Create a tailored resume version

Goal: Create a job-specific resume version. **Type:** Hybrid.

Input / preconditions: Base resume, confirmed profile, reviewed job and accepted edits; all source records exist.

GUI: Select job > Tailor Resume; review edits. **CLI:** career resume tailor --job J1

Workflow: 1. DocumentService.tailorResume() loads sources and calls Resume.deepCopy(). 2. LLMClient.generate() proposes evidence-supported changes. 3. The user selects or edits changes; saveDraft() validates and saves the reviewed copy.

Alternatives/errors: Flag unsupported claims for correction. Cancellation discards the copy; the base resume stays unchanged.

Output / postconditions: A separate reviewed Resume version is saved. **Sequence:** SD03, SD08.

F07 / UC07 - Draft and revise a cover letter

Goal: Draft a cover letter for the selected job. **Type:** Hybrid.

Input / preconditions: Existing confirmed profile and reviewed job, chosen tone and optional motivation.

GUI: Select job > Cover Letter; edit and Save. **CLI:** career letter draft --job J1 --tone professional

Workflow: 1. DocumentService.draftLetter() loads source facts. 2. LLMClient.generate() returns a draft and evidence references. 3. The user checks facts and edits the preview; saveDraft() stores the approved letter.

Alternatives/errors: Omit unknown employer details and correct unsupported achievements. Failure preserves any earlier draft.

Output / postconditions: Reviewed CoverLetter is saved, or preview remains unsaved. **Sequence:** SD03, SD08.

F08 / UC08 - Generate targeted interview questions

Goal: Generate targeted practice questions. **Type:** Hybrid.

Input / preconditions: Existing confirmed profile/job; count 1-20 and technical or behavioural focus.

GUI: Interview > Generate Questions. **CLI:** career interview questions --job J1 --count 8

Workflow: 1. InterviewService.generateQuestions() validates the count and loads sources. 2. LLMClient.generate() builds questions linked to requirements. 3. The service validates IDs, count and links, then saves the set.

Alternatives/errors: Reject invalid counts. Repair malformed/duplicate questions once; do not save an incomplete set.

Output / postconditions: QuestionSet is saved for preview or reuse in practice. **Sequence:** SD04, SD08.

2. Features and use cases: F09-F12

Actor: Applicant; AI provider supports hybrid features. Trigger: the GUI action or CLI command shown. Fxx maps to UCxx. The numbered workflow is the main success scenario and explains class collaboration.

F09 / UC09 - Conduct a mock interview and receive feedback

Goal: Practise answers and receive feedback. **Type:** Hybrid.

Input / preconditions: Existing non-empty QuestionSet and answers entered one at a time.

GUI: Interview > Start; answer, pause or continue. **CLI:** career interview start --questions Q1

Workflow: 1. InterviewService.startInterview() loads questions and creates a session. 2. submitAnswer() saves each answer and calls LLMClient.generate() for feedback. 3. It saves feedback and advances the index until paused or complete.

Alternatives/errors: Reject blank answers or completed sessions. If feedback fails, retain the answer as pending for retry.

Output / postconditions: InterviewSession, answers and progress persist; feedback is coaching. **Sequence:** SD04, SD08.

F10 / UC10 - Track applications, notes and deadlines

Goal: Track application status and deadlines. **Type:** Deterministic.

Input / preconditions: Existing job ID, status, notes, optional deadline and list filters.

GUI: Applications > Add/Edit or filter records. **CLI:** career application update --job J1 --status applied

Workflow: 1. ApplicationService.updateApplication() loads the job/record and validates dates, status and revision. 2. WorkspaceRepository.save() updates it. 3. listApplications() filters records and computes overdue flags from the local date.

Alternatives/errors: Reject invalid dates/statuses. Unknown deadlines stay blank; reuse the existing job record.

Output / postconditions: ApplicationRecord is saved; the user explicitly sets its status. **Sequence:** SD05.

F11 / UC11 - Prepare an application action plan

Goal: Plan the work needed for an application. **Type:** Hybrid / agent.

Input / preconditions: Existing confirmed profile/job, available minutes and optional deadline; AI access.

GUI: Select job > Prepare Application; review plan. **CLI:** career plan prepare --job J1 --minutes 120

Workflow: 1. PlanService.prepareApplication() starts CareerAgent.run(). 2. The agent chooses tools; ToolRegistry binds validated InvokeToolCommands and returns observations. 3. PlanValidator checks tasks/time. The user accepts savePlan() and separately saves selected document drafts.

Alternatives/errors: Reject invalid tools; stop at 8 calls/60 seconds. Ask about missing evidence or report an infeasible plan.

Output / postconditions: Reviewed PreparationPlan is saved, or a clarification/incomplete result is returned. **Sequence:** SD06, SD08.

F12 / UC12 - Export a reviewed application package

Goal: Export reviewed application documents. **Type:** Deterministic.

Input / preconditions: Reviewed resume or letter, optional plan, chosen format and writable new output path.

GUI: Export > Select documents and format. **CLI:** career export --job J1 --format markdown --out ./J1

Workflow: 1. ExportService.exportPackage() loads reviewed versions. 2. ApplicationPackageDirector.construct() calls reset(), selected add methods and build() on a Markdown or text builder. 3. FileExporter.write() writes the completed package atomically.

Alternatives/errors: Reject unreviewed documents/existing destinations. Remove temporary output on failure; list omitted optional files.

Output / postconditions: ApplicationPackage and manifest are written locally; nothing is submitted. **Sequence:** SD07.

3. Design patterns

Facade

Classes and roles: CareerAssistantFacade; CareerGUI, CareerCLI and the eight services.

The two interfaces need the same feature operations. execute(FeatureRequest) routes requests and translates results without absorbing service algorithms. Without it, GUI and CLI duplicate coordination and error handling. The facade must stay thin.

Adapter

Classes and roles: LLMClient (target), OpenAILLMAdapter (adapter), ProviderClient (adaptee).

The provider has its own request and response formats. generate(LLMRequest) translates our contract to and from those formats. Without the adapter, provider details spread across services. The cost is maintaining a separate internal contract.

Decorator

Classes and roles: LLMClient (component), OpenAILLMAdapter (concrete component), LoggingLLMClient (decorator).

LoggingLLMClient wraps one LLMClient, records timing/outcome and delegates the same request. It preserves return values and errors. Without it, logging is duplicated or mixed into provider code. It adds one delegation and must never mask failures.

Prototype

Classes and roles: ResumePrototype (interface), Resume (prototype), DocumentService (client).

tailorResume() calls deepCopy() before changing content. The copy duplicates mutable sections/entries, receives a new ID and starts unreviewed. Without it, shared lists can accidentally change the base resume. The copy logic must be maintained when fields change.

Builder

Classes and roles: ApplicationPackageDirector; ApplicationPackageBuilder; MarkdownPackageBuilder and TextPackageBuilder; ApplicationPackage.

The director calls reset(), selected add methods and build(). The two builders produce different representations of the same selected documents. FileExporter handles writing separately. Without it, format conditions mix with assembly. Extra classes are justified by optional parts and two output formats.

Command

Classes and roles: ToolRegistry (invoker), ToolCommand, InvokeToolCommand (concrete command), AgentTool (receiver), CareerAgent (client).

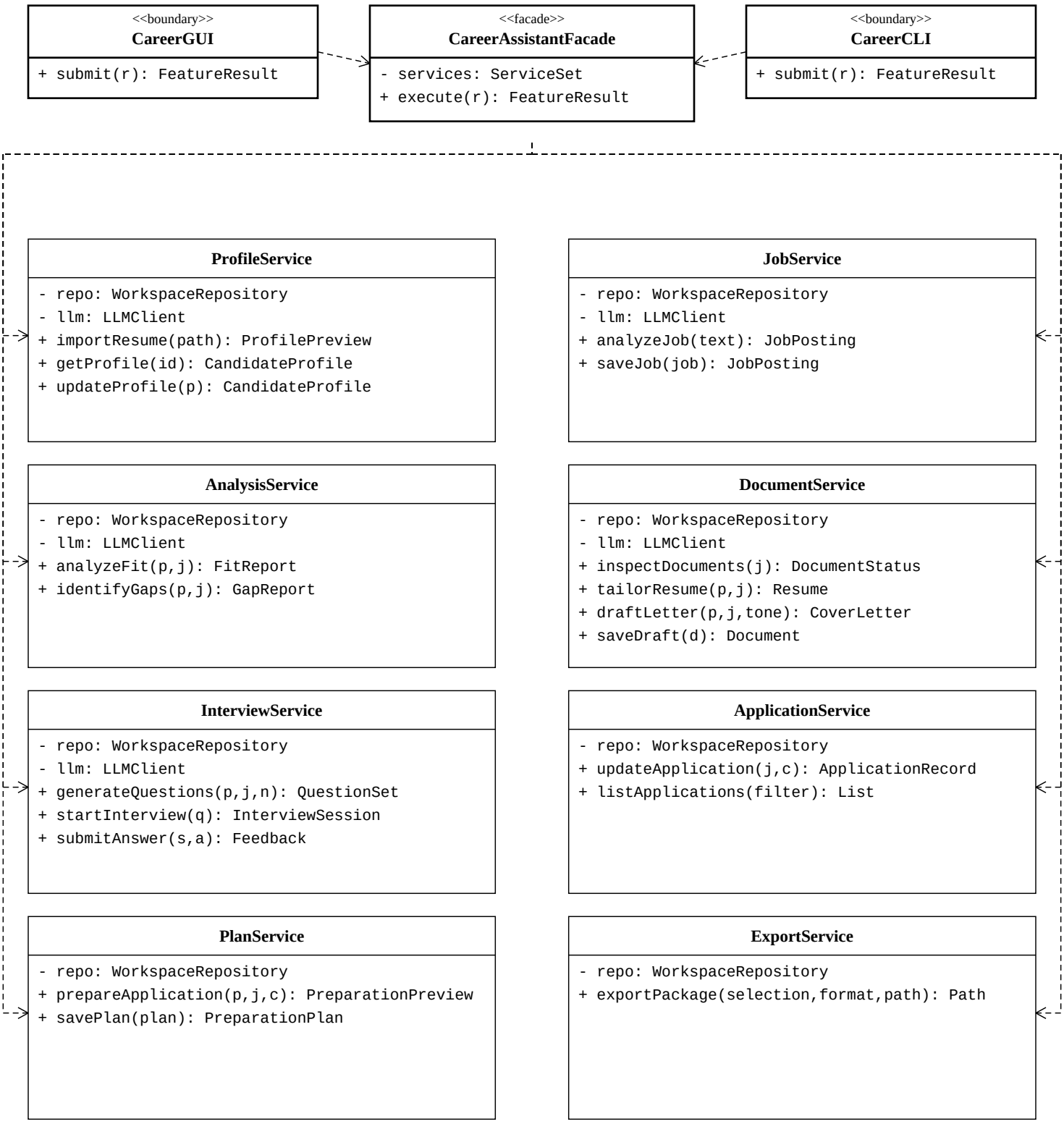
A validated call is bound to a receiver, arguments and call ID in an InvokeToolCommand. execute() invokes that request and provides a clear unit for tracing/testing. AgentTool alone is not counted as Command. Without the bound object, request data and execution become mixed into the agent loop. Undo is unnecessary for observations and unsaved previews.

Factory Method

Classes and roles: ResumeImporter (abstract creator), PdfResumeImporter/TxtResumeImporter (concrete creators), DocumentParser and its two products.

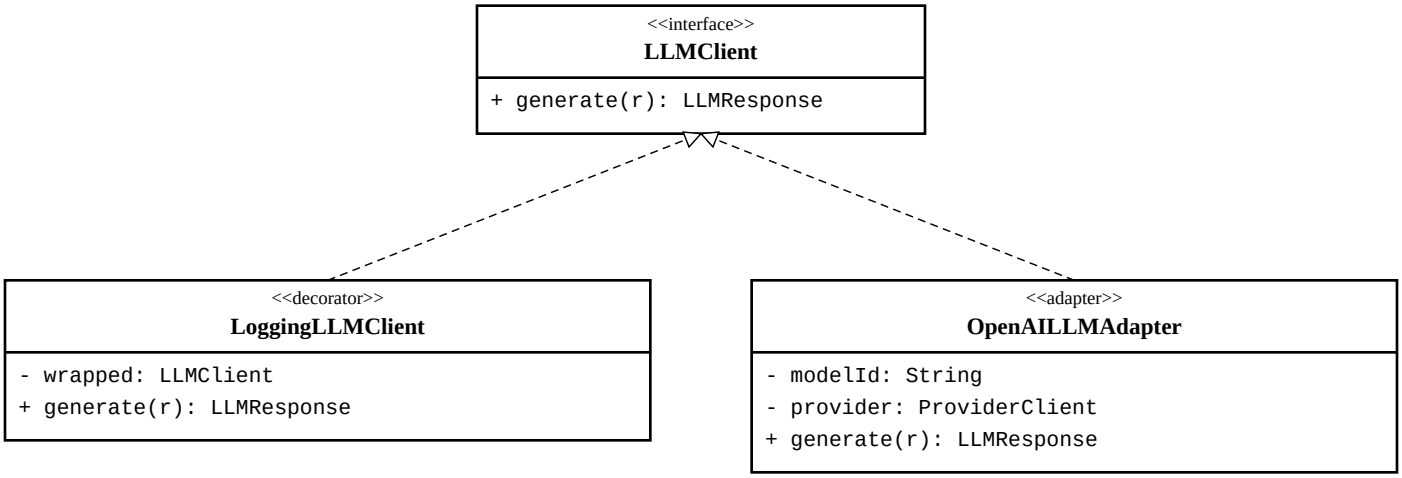
importText() calls the abstract createParser(), overridden to return PdfTextParser or PlainTextParser. ProfileService selects a creator from a format map. The override is the Factory Method; a file-extension switch alone would not be enough. New formats add classes rather than modifying the import workflow, at the cost of extra creator classes.

4. Class diagram A - application services

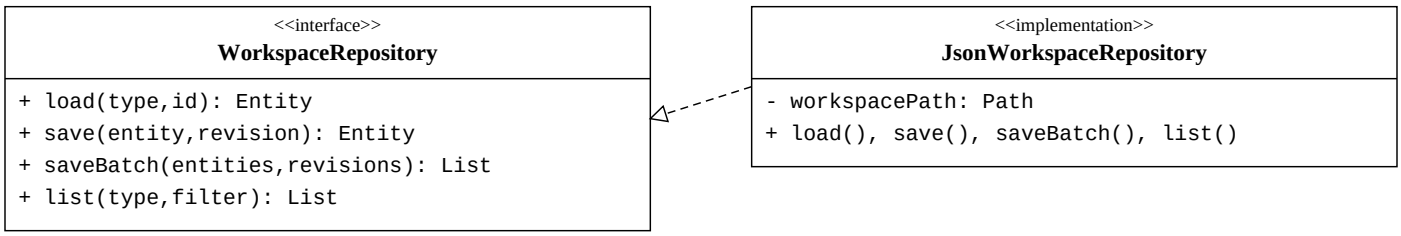


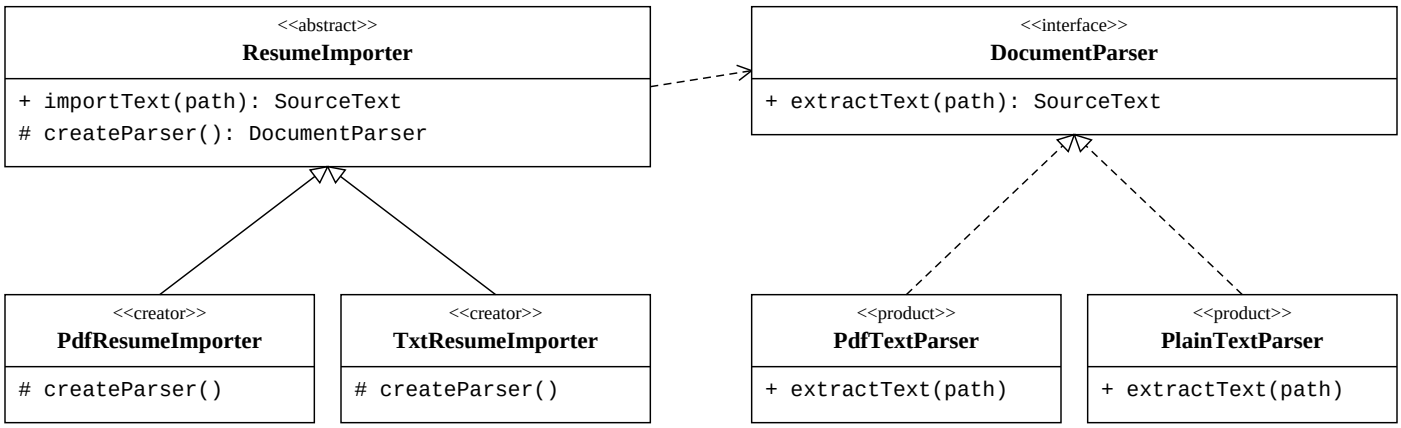
The facade depends on every service shown. Services hold injected repository/model interfaces; PlanService also holds CareerAgent, while ExportService holds a director and file writer. These field types are defined in the next views and data dictionary. p/j/c abbreviate profileId/jobId/constraints.

4. Class diagram B - AI, storage and parsing



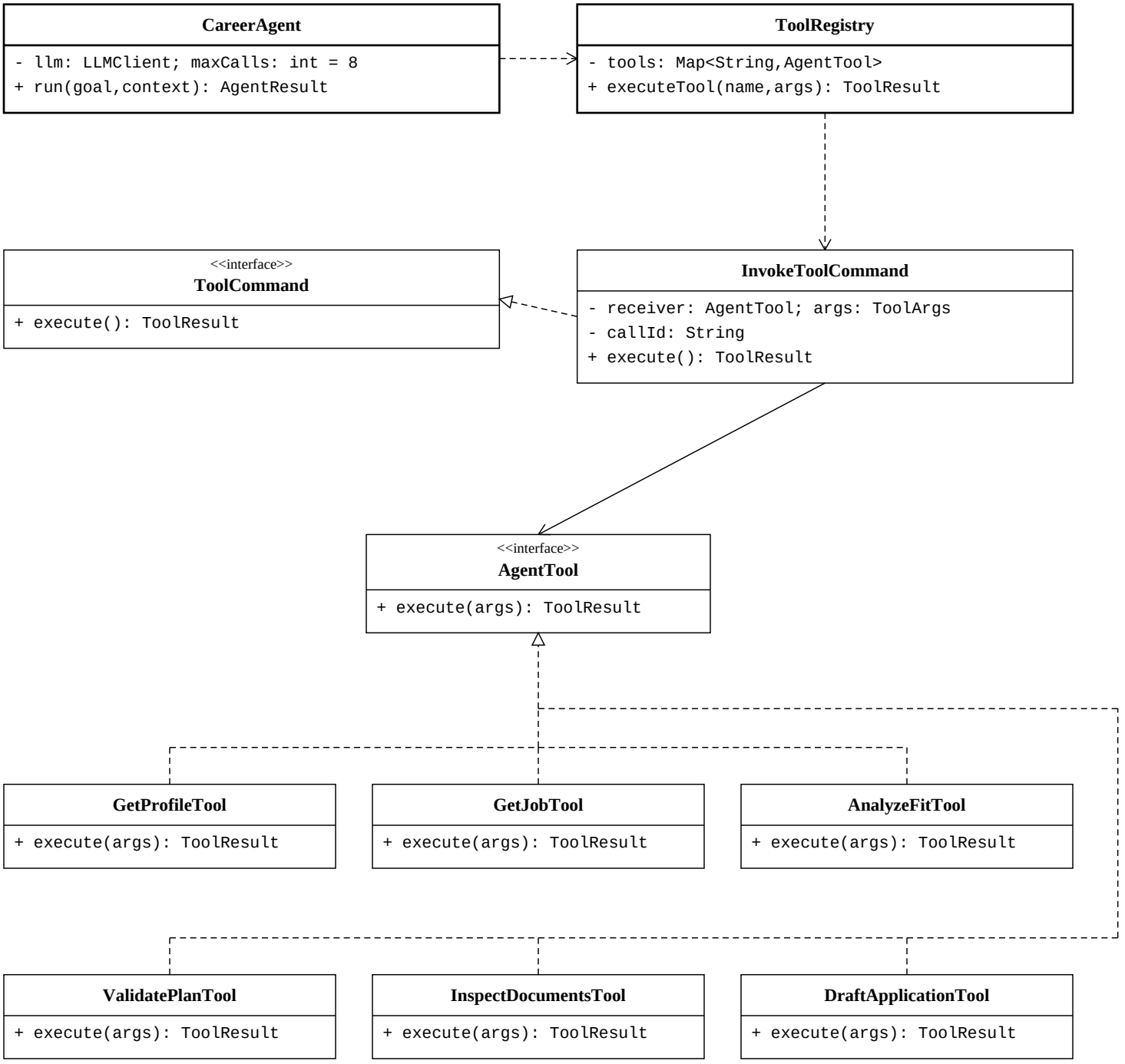
LoggingLLMClient holds 1 wrapped LLMClient; the adapter uses ProviderClient.





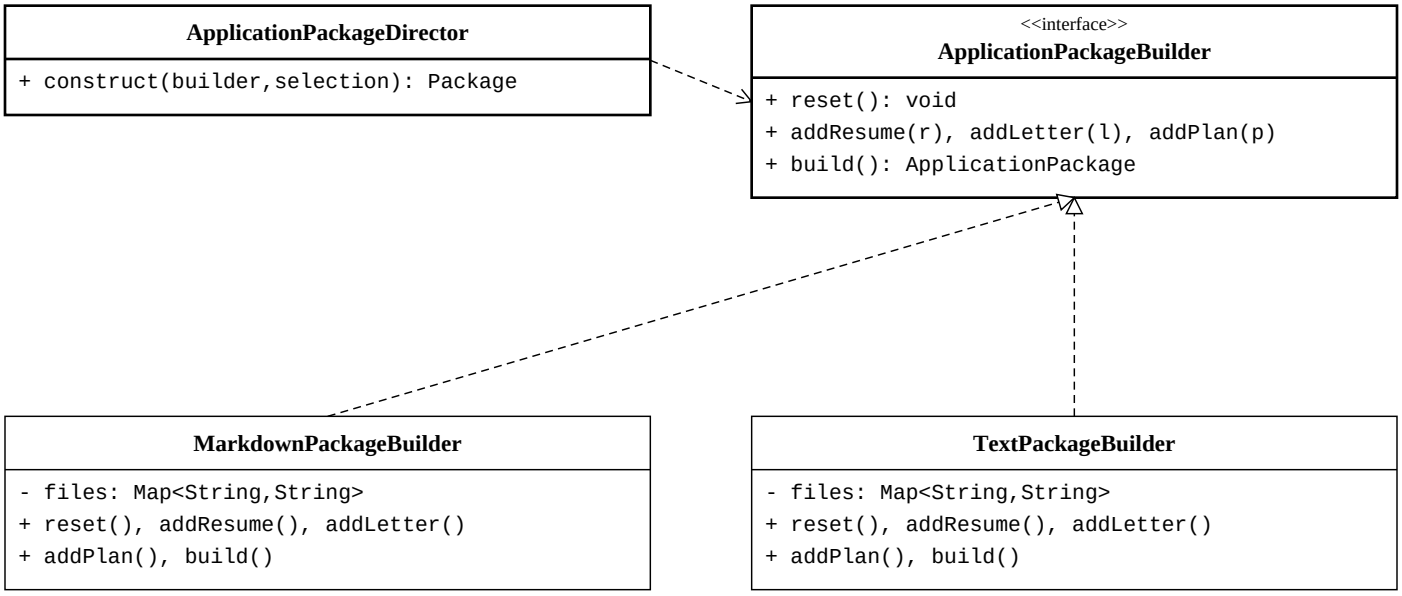
PdfResumeImporter creates PdfTextParser; TxtResumeImporter creates PlainTextParser. ProfileService selects the creator from a format map, then calls importText(). Concrete operations have the same return types as their interfaces. Dashed hollow triangles mean realization; solid hollow triangles mean inheritance.

4. Class diagram C - agent and commands

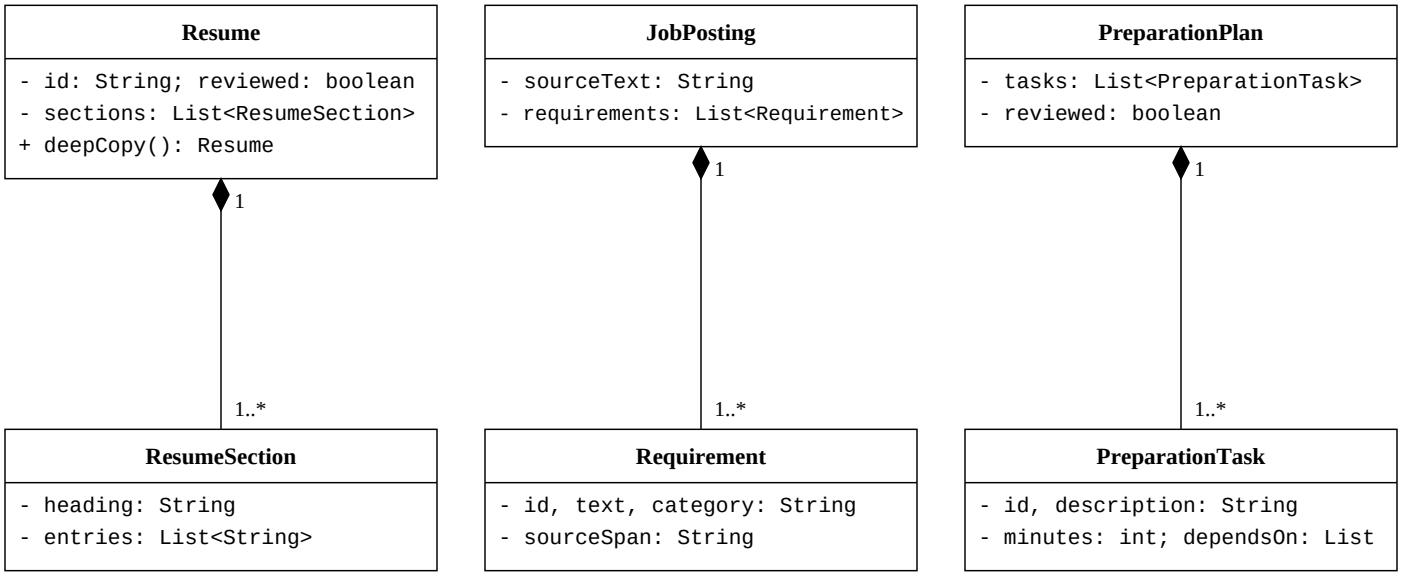


ToolRegistry binds validated requests to InvokeToolCommand; each command has one receiver. Each of the six concrete receivers implements AgentTool. GetProfile/GetJob retrieve records; AnalyzeFit uses AnalysisService; ValidatePlan uses PlanValidator; the two document tools use DocumentService. PlanValidator.validate(plan,constraints): ValidationResult checks IDs, cycles and available time.

4. Class diagram D - export and owned data

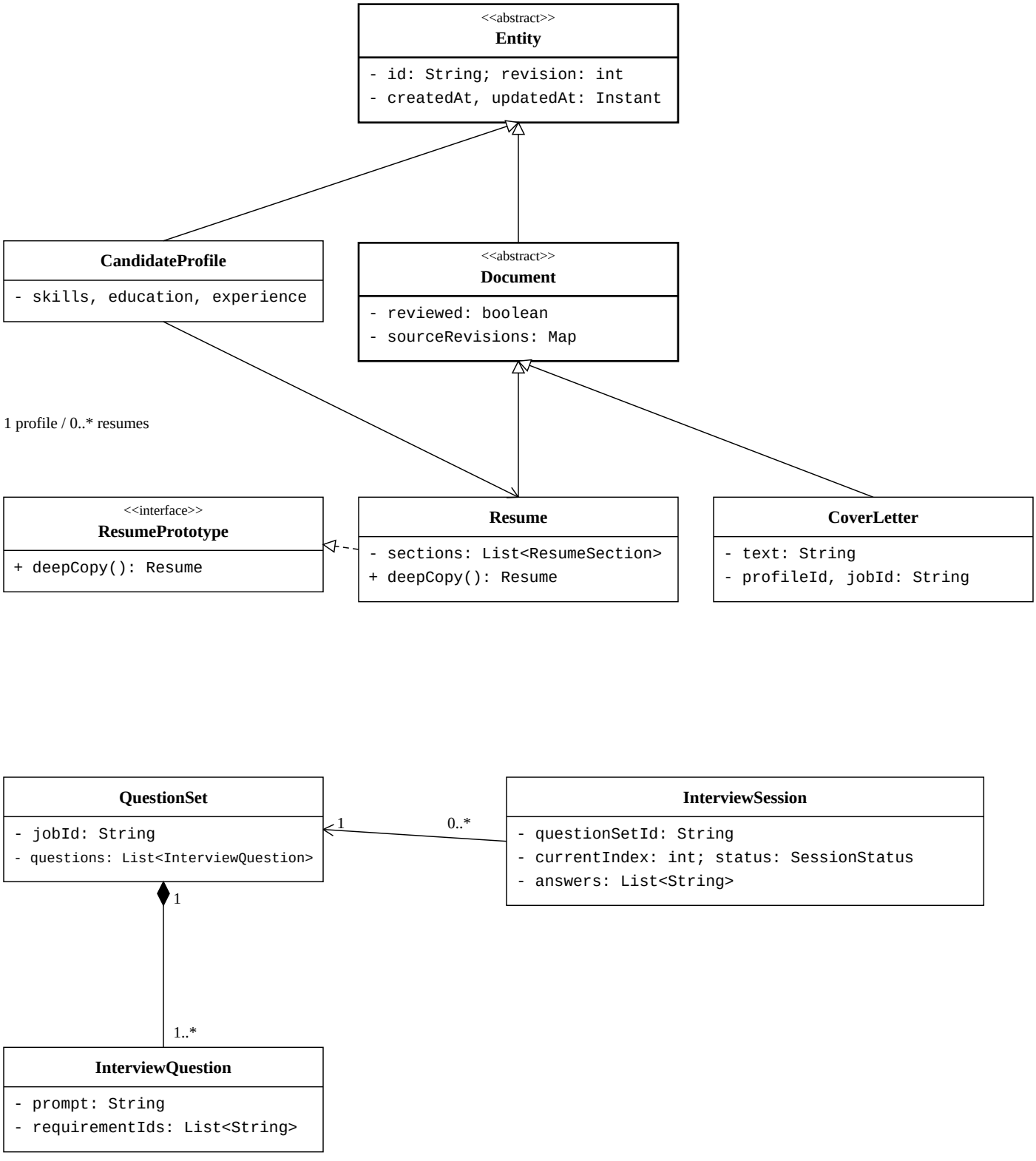


ExportService uses the director, repository and FileExporter.write(package,path). Builders create ApplicationPackage (files and sourceRevisions maps). All add methods return void; build() returns the product. Package above abbreviates ApplicationPackage.



Filled diamonds show composition. Resume implements ResumePrototype.deepCopy(): Resume (Prototype). One profile is associated with 0..* resume versions. Each version records its source resume ID. A QuestionSet similarly owns 1..* InterviewQuestions, and one set may have 0..* InterviewSessions. Remaining record fields and responsibilities are listed next.

4. Class diagram E - documents and records



A resume is both a Document and a ResumePrototype. Each saved resume belongs to one candidate profile. A question set owns its questions and can be reused in several interview sessions. Entity supplies the common ID, revision and timestamps.

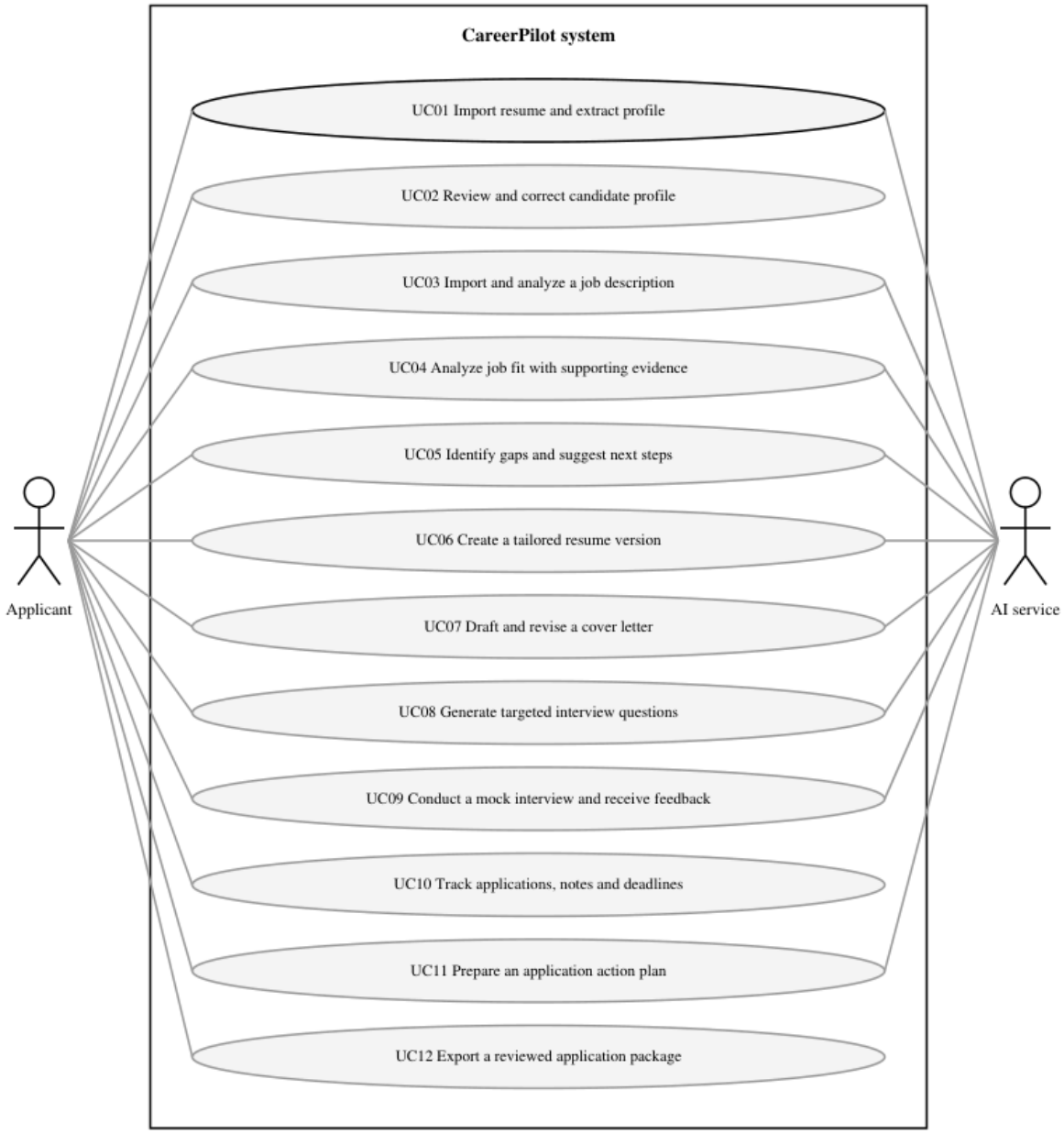
4. Supporting classes and data dictionary

Class / interface	Fields or responsibility
CandidateProfile	Entity containing confirmed skills, education and experience. Owns candidate facts used by analysis and drafting.
FitReport / GapReport	Source revisions and evidence-linked matches / next-step actions. Each fit report references one profile and one job; each gap report references one fit report.
CoverLetter / Document	CoverLetter stores text and job/profile IDs. Abstract Document adds reviewed and sourceRevisions. Resume and CoverLetter extend Document.
ApplicationRecord	jobId, status, notes and optional deadline; at most one record per job. Tracks explicit user updates.
QuestionSet / InterviewQuestion	Set stores jobId and questions. Question stores prompt, requirement IDs and evaluation criteria.
InterviewSession / Feedback	Session stores questionSetId, currentIndex, status and answers. Feedback stores question/answer comments; failed feedback stays pending.
Entity / EvidenceRef	Entity provides id, revision, createdAt and updatedAt. EvidenceRef identifies a profile field or source span at a specific revision.
FeatureRequest / FeatureResult	Operation, arguments, expected revision / status, payload, warnings and error. Shared by GUI and CLI.
LLMRequest / LLMResponse	Messages, schema and permitted tools / structured data or requested ToolCalls.
ToolCommand / AgentTool	Bound request execution / approved tool receiver contract. ToolResult contains success, data or an error.
AgentResult / PreparationPreview	Status, plan, unsaved drafts and optional question / plan, selected draft previews and warnings for user review.
SourceText / ProfilePreview	Extracted text and source locations / candidate draft, original resume and validation warnings.
DocumentStatus / ValidationResult	Artifact IDs and missing/stale/reviewed flags / valid, feasible, errors and warnings.
Message / ToolDefinition / ToolCall	Role and content / name and argument schema / call ID, name and arguments.
RunContext / Constraints / ToolArgs	Source IDs and observations / available minutes, deadline, preferences / validated argument map.
ServiceSet / Selection / ProviderClient	Injected services / selected document IDs and optional plan / external model SDK or HTTP client.
Enums and Java types	Operation, ResultStatus and ResponseKind enumerate operations/outcomes. Application Status and SessionStatus describe state. String, int, boolean, List, Map, Path, LocalDate and Instant use Java meanings.

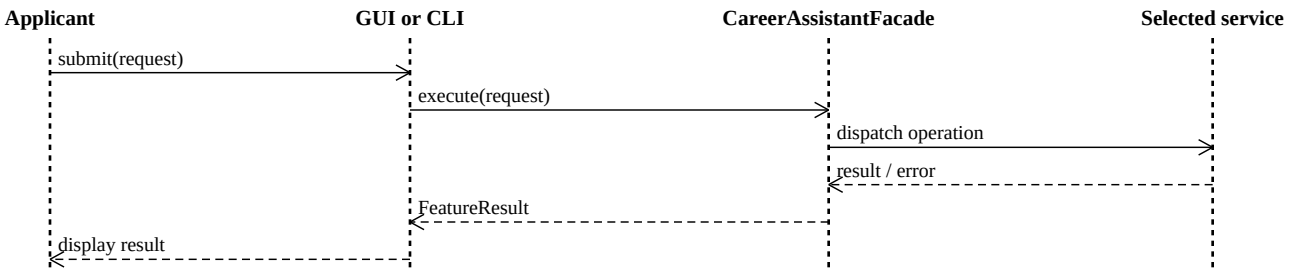
The class views describe one system. Repeated class names refer to the same class. The supporting PlantUML file provides the combined model. Public methods use +; private fields use -; protected operations use #. DTOs carry data and do not add hidden behaviour.

5. Use-case diagram and interface flow

The applicant accesses all 12 use cases through either interface. The AI provider supports hybrid features. Detailed goals, inputs, preconditions, main flows, alternatives and postconditions are in Section 2.



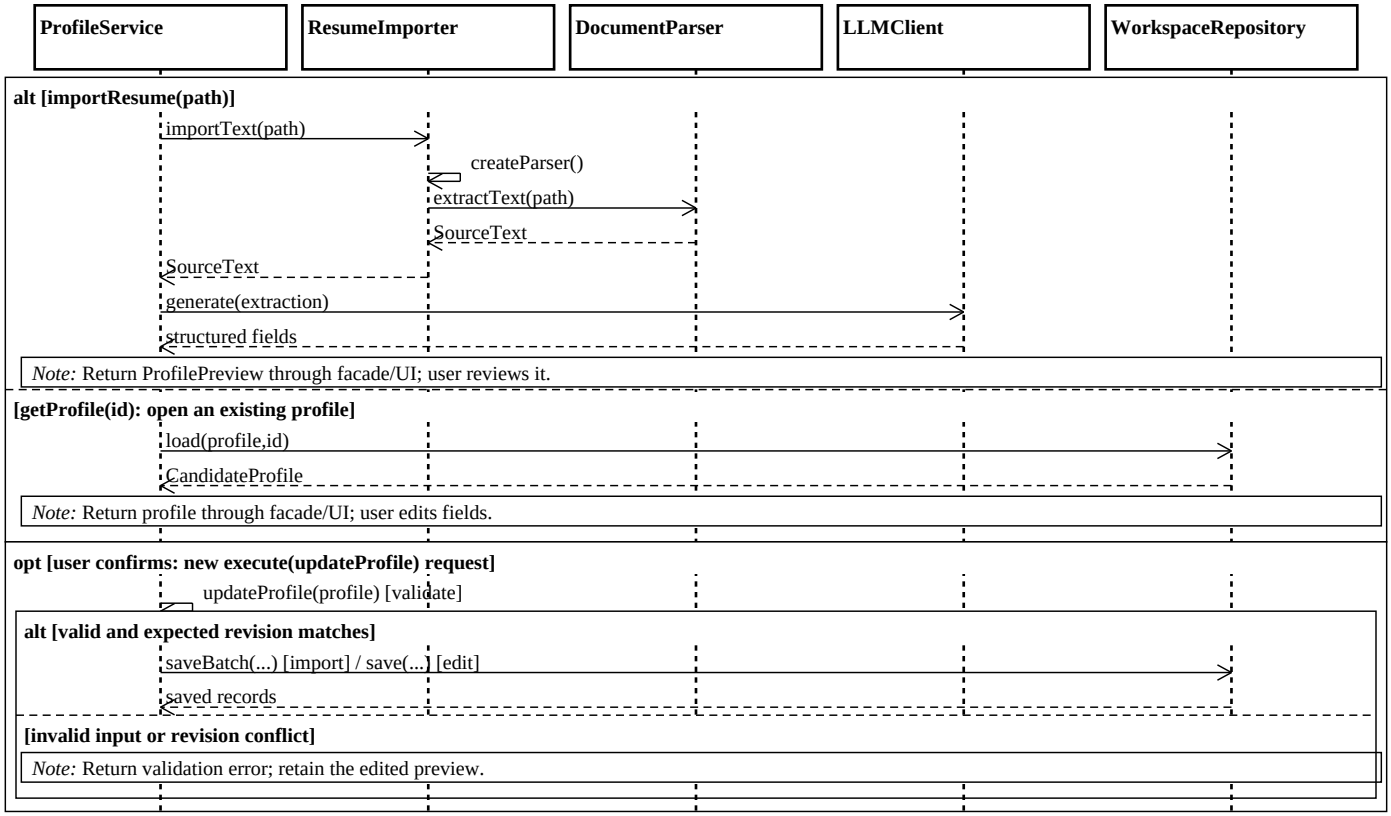
Shared request path for SD01-SD08: the selected service performs the operation shown in each sequence. Every confirmation/save is a separate request. GUI and CLI are system boundaries.



6. Sequence diagrams: SD01 and SD05

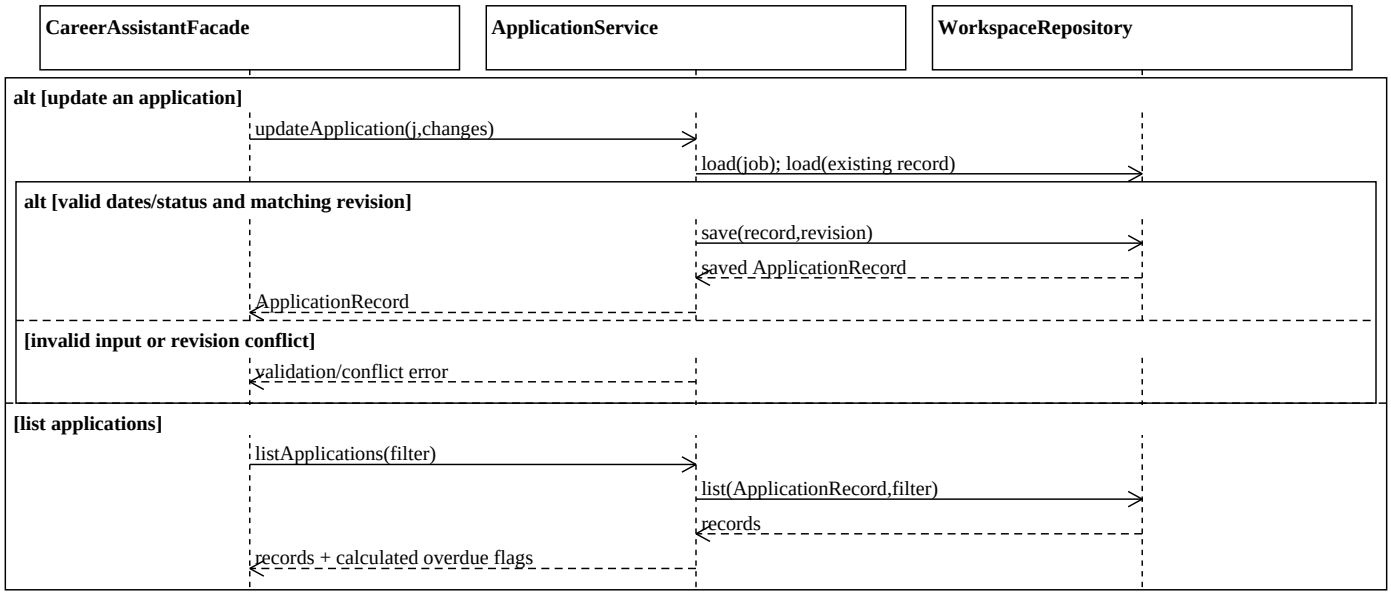
Shared actor, GUI/CLI and facade entry/return: Section 5. Each save/confirmation starts a new request.

SD01 - Import and edit a profile



Import returns a preview. Confirmation calls updateProfile() separately; saveBatch() stores the imported profile and resume atomically.

SD05 - Application tracking

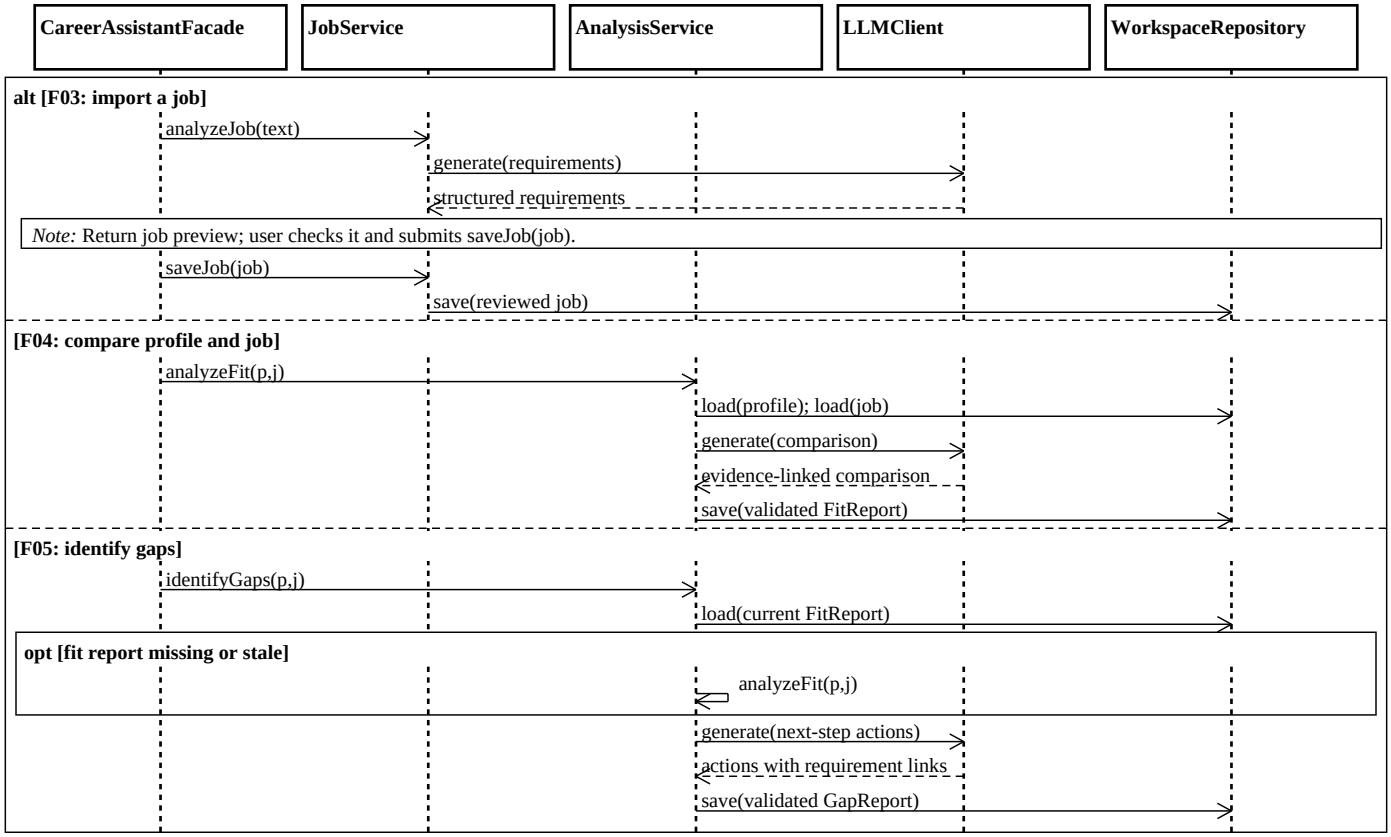


Status changes are explicit user updates. Unknown deadlines stay blank; the system does not submit applications.

6. Sequence diagrams: SD02 and SD03

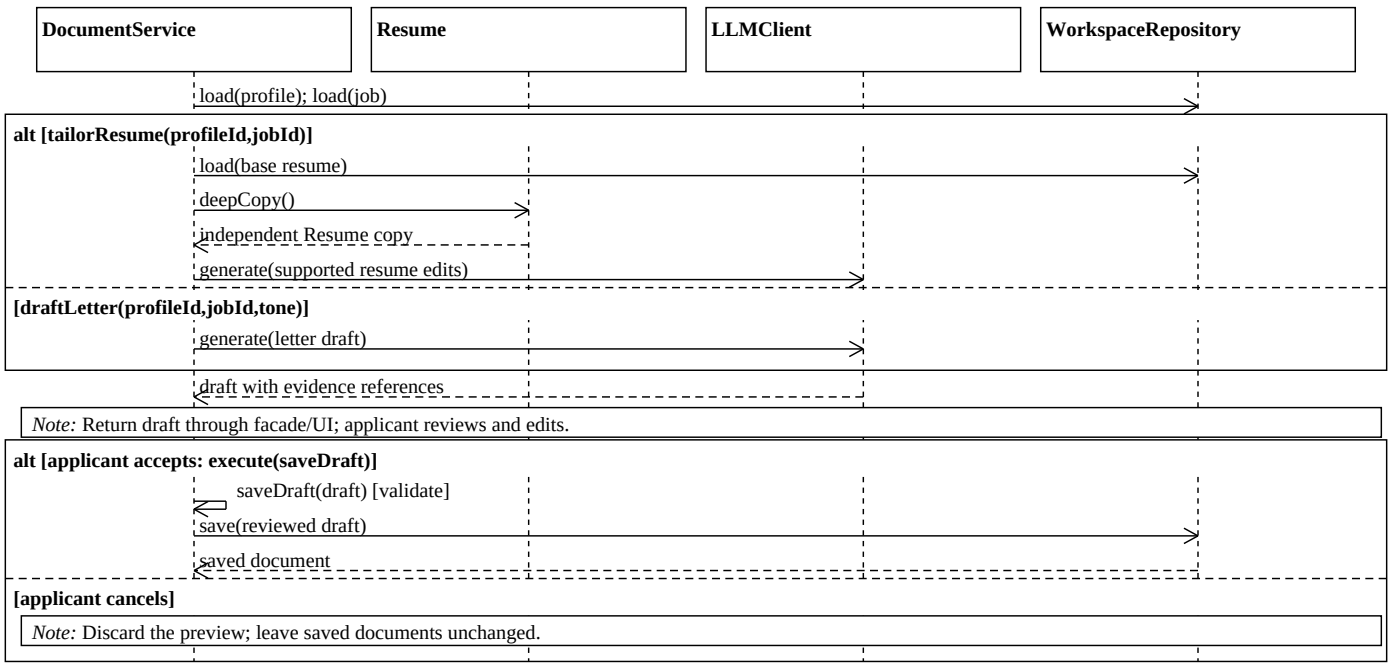
Shared actor, GUI/CLI and facade entry/return: Section 5. Each save/confirmation starts a new request.

SD02 - Job analysis, fit and gaps



These are separate user requests. Invalid evidence gets one repair attempt, then an error. p/j mean profileId/jobId.

SD03 - Resume and cover-letter drafts

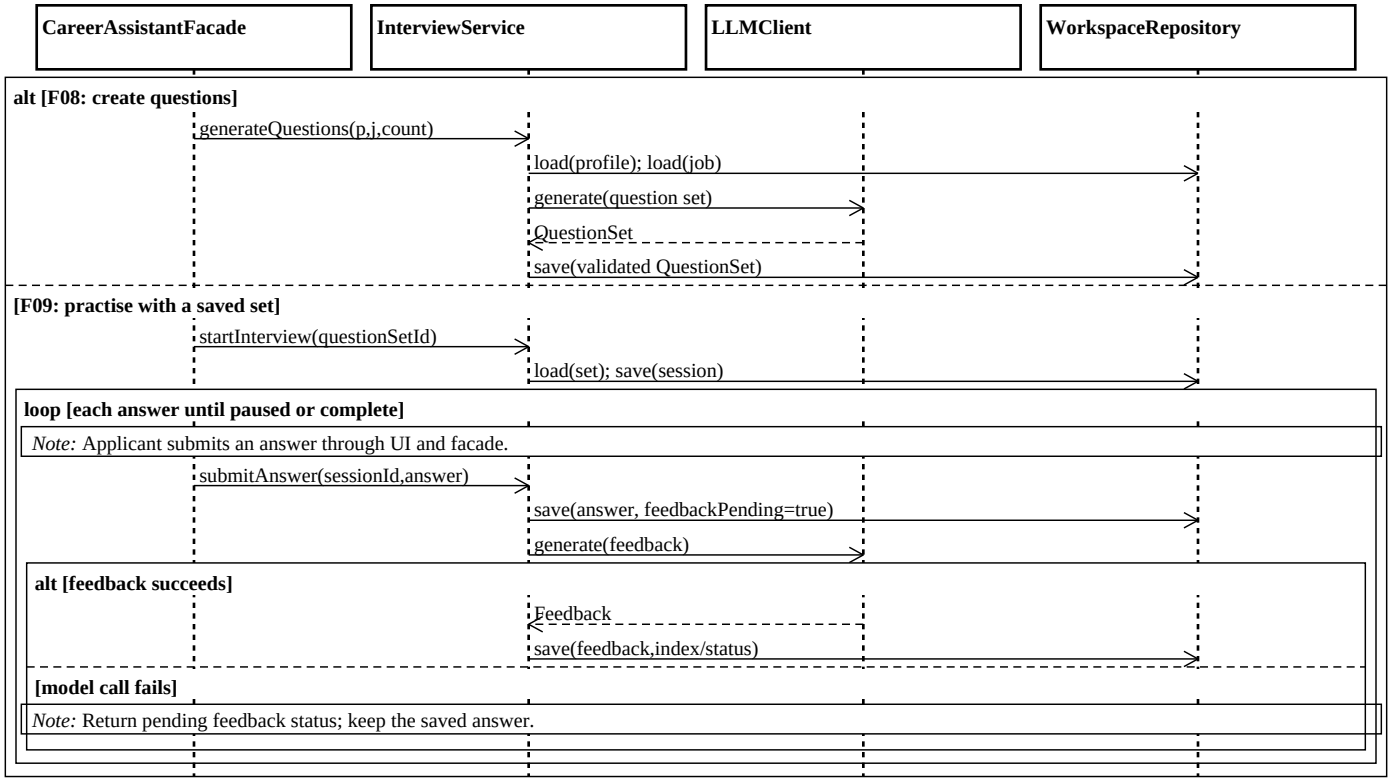


Generation returns a preview. Unsupported claims require correction before saveDraft(); cancellation leaves saved documents unchanged.

6. Sequence diagrams: SD04 and SD07

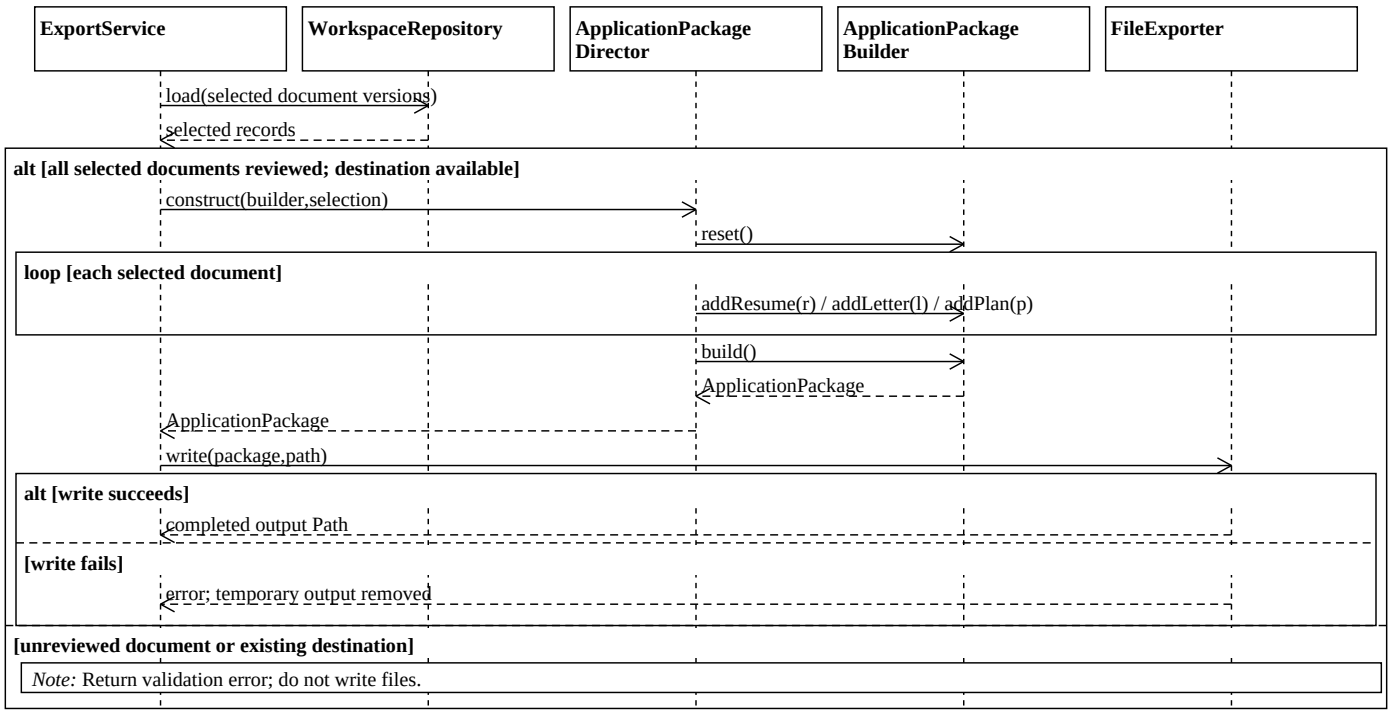
Shared actor, GUI/CLI and facade entry/return: Section 5. Each save/confirmation starts a new request.

SD04 - Interview questions and practice



Reject blank answers and invalid counts. Users can retry pending feedback or continue; saved progress survives closing the app.

SD07 - Build and export a package

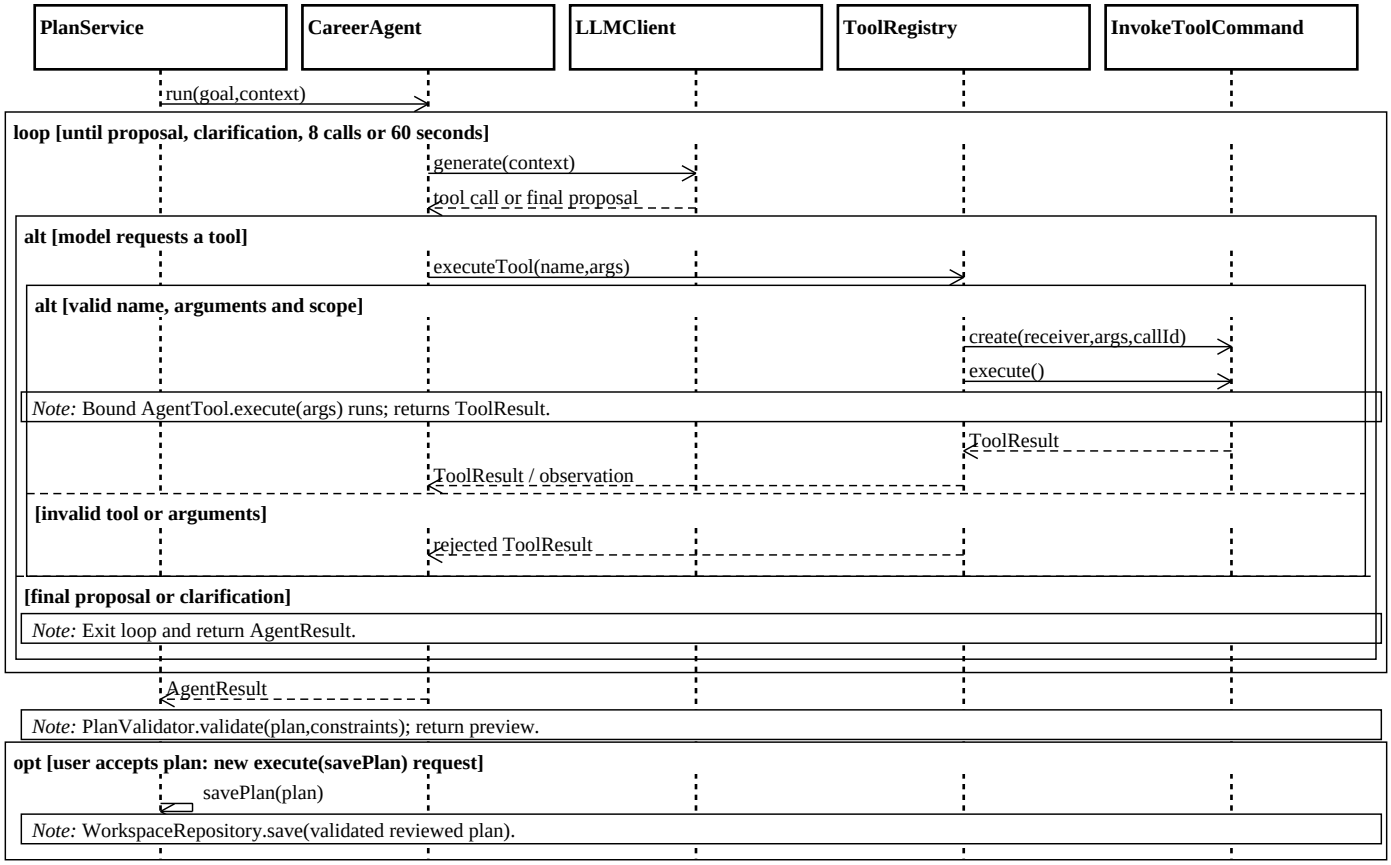


The selected builder produces Markdown or text. FileExporter writes temporary output and renames it only when complete.

6. Sequence diagrams: SD06

Shared actor, GUI/CLI and facade entry/return: Section 5. Each save/confirmation starts a new request.

SD06 - Application preparation agent

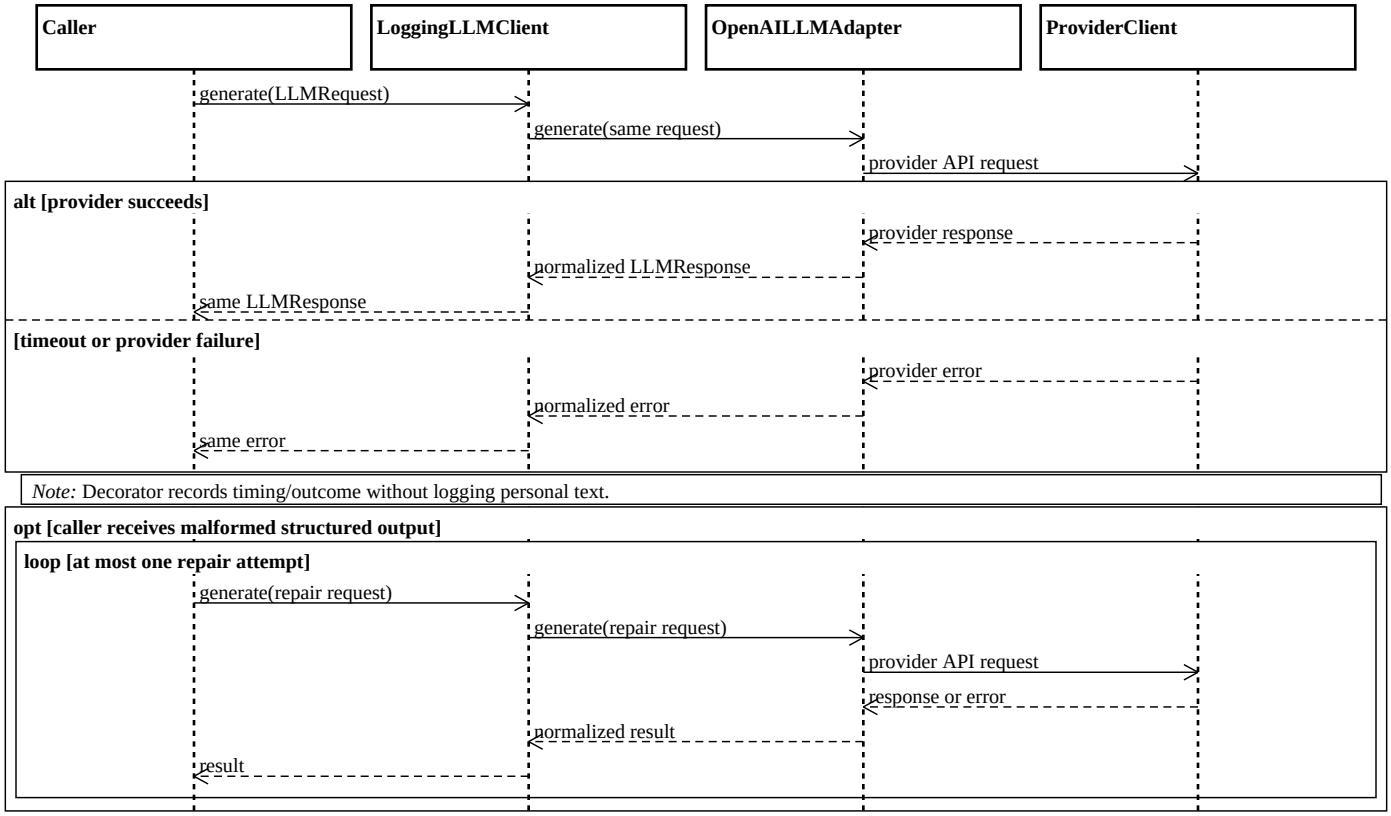


Tool results guide later decisions. PlanService validates the final plan; accepted drafts are saved separately through DocumentService (SD03).

6. Sequence diagrams: SD08

Shared actor, GUI/CLI and facade entry/return: Section 5. Each save/confirmation starts a new request.

SD08 - Model call and logging



Caller is the service or agent using LLMClient. Schema/evidence checks belong to the caller; logging preserves results and errors.

7. Feature-to-design mapping: F01-F06

All features enter through CareerGUI/CareerCLI.submit() and CareerAssistantFacade.execute(). GUI and CLI share the same services. Service responsibilities and interactions are described in each feature workflow.

ID / use case	Description / type	Classes	Key methods	Sequence	Patterns
F01 / UC01	Import resume and extract profile; Hybrid	ProfileService, DocumentParser, PdfTextParser, CandidateProfile, Resume, LLMClient, WorkspaceRepository, ResumeImporter, PdfResumeImporter, TxtResumeImporter, PlainTextParser	importResume(), extractText(), generate(), updateProfile(), save(), importText(), createParser()	SD01, SD08	Facade, Adapter, Decorator, Factory Method
F02 / UC02	Review and correct candidate profile; Deterministic	ProfileService, CandidateProfile, WorkspaceRepository	getProfile(), updateProfile(), load(), save()	SD01	Facade
F03 / UC03	Import and analyze a job description; Hybrid	JobService, JobPosting, Requirement, LLMClient, WorkspaceRepository	analyzeJob(), generate(), saveJob(), save()	SD02, SD08	Facade, Adapter, Decorator
F04 / UC04	Analyze job fit with supporting evidence; Hybrid	AnalysisService, FitReport, CandidateProfile, JobPosting, LLMClient, WorkspaceRepository	analyzeFit(), load(), generate(), save()	SD02, SD08	Facade, Adapter, Decorator
F05 / UC05	Identify gaps and suggest next steps; Hybrid	AnalysisService, FitReport, GapReport, LLMClient, WorkspaceRepository	identifyGaps(), analyzeFit(), generate(), save()	SD02, SD08	Facade, Adapter, Decorator
F06 / UC06	Create a tailored resume version; Hybrid	DocumentService, Resume, ResumeSection, LLMClient, WorkspaceRepository	tailorResume(), deepCopy(), generate(), saveDraft(), save()	SD03, SD08	Facade, Prototype, Adapter, Decorator

The workflows in Section 2 explain how these classes collaborate. All requests use the shared interface flow in Section 5.

7. Feature-to-design mapping: F07-F12

All features enter through CareerGUI/CareerCLI.submit() and CareerAssistantFacade.execute(). GUI and CLI share the same services. Service responsibilities and interactions are described in each feature workflow.

ID / use case	Description / type	Classes	Key methods	Sequence	Patterns
F07 / UC07	Draft and revise a cover letter; Hybrid	DocumentService, CoverLetter, LLMClient, WorkspaceRepository	draftLetter(), generate(), saveDraft(), save()	SD03, SD08	Facade, Adapter, Decorator
F08 / UC08	Generate targeted interview questions; Hybrid	InterviewService, QuestionSet, InterviewQuestion, LLMClient, WorkspaceRepository	generateQuestions(), generate(), save()	SD04, SD08	Facade, Adapter, Decorator
F09 / UC09	Conduct a mock interview and receive feedback; Hybrid	InterviewService, InterviewSession, QuestionSet, LLMClient, WorkspaceRepository	startInterview(), submitAnswer(), generate(), save()	SD04, SD08	Facade, Adapter, Decorator
F10 / UC10	Track applications, notes and deadlines; Deterministic	ApplicationService, ApplicationRecord, WorkspaceRepository	updateApplication(), listApplications(), load(), save(), list()	SD05	Facade
F11 / UC11	Prepare an application action plan; Hybrid / agent	PlanService, CareerAgent, ToolRegistry, AgentTool, PlanValidator, PreparationPlan, LLMClient, ToolCommand, InvokeToolCommand, InspectDocumentsTool, DraftApplicationTool, DocumentService, PreparationPreview	prepareApplication(), run(), executeTool(), execute(), validate(), savePlan(), inspectDocuments(), tailorResume(), draftLetter()	SD06, SD08	Facade, Adapter, Decorator, Command
F12 / UC12	Export a reviewed application package; Deterministic	ExportService, ApplicationPackageDirector, ApplicationPackageBuilder, MarkdownPackageBuilder, TextPackageBuilder, ApplicationPackage, FileExporter	exportPackage(), construct(), reset(), addResume(), addLetter(), addPlan(), build(), write()	SD07	Facade, Builder

The workflows in Section 2 explain how these classes collaborate. All requests use the shared interface flow in Section 5.

8. References and next steps

Stage 3 will use KUMA to evaluate agent behaviour [R1]. The CLI will provide structured results and a record of tool calls for these tests. The exact connection to KUMA will be decided during implementation.

Planned checks include missing qualifications, stale documents and unrealistic deadlines. JUnit tests will cover Java behaviour such as copying resumes, validating records and handling failed exports.

References

[R1] EECS3311 Course Project: Design, Build, and Test an AI Agent. Project-Instruction.pdf.

[R2] EECS3311 Fall 2026 Course Project Stage-1 Instruction. stage1.pdf.

[R3] Song Wang. OOP Design Principles. EECS3311-1- OOP Design.pdf.

[R4] JUnit lecture. EECS3311-2-Junit.pdf.

[R5] Song Wang. UML. EECS3311-3-UML.pdf.

[R6] Song Wang. UML II. EECS3311-4-UML-II.pdf.

[R7] Song Wang. SOLID Design Principles. EECS3311-5-SOLID.pdf.

[R8] Song Wang. Creational Design Patterns. EECS3311-6-Design Pattern-Creational .pdf.

[R9] Song Wang. Structural Patterns. EECS3311-07-Structural Patterns.pdf.

[R10] OpenAI. GPT-4.1 mini model documentation. developers.openai.com/api/docs/models/gpt-4.1-mini. Checked October 9, 2026.

AI assistance

I used ChatGPT to help draft and revise the feature descriptions, UML structure, design-pattern explanations and formatting. This report describes the proposed design; implementation and testing belong to the later stages.